AI AGENT 应用创新赛 • MVP 参赛方案

# 智投罗盘

## A 股 AI 智能选股 · 3 天可开发版（前后端分离）

AI 对话 • A股多因子 • 可视化调参

| | |
|---|---|
| **文档版本** | V1.2（A股版） |
| **文档类型** | MVP 设计说明书 + 3 天开发计划 |
| **项目底座** | 现有 Python 量化计算框架（复用因子/筛选/组合/回测结构） |
| **新增范围** | AI 对话画像 + A股10因子 + 解读追问 + 可视化调参 |
| **开发周期** | 3 天（D1 数据与画像 / D2 选股解读 / D3 调参可视化） |
| **技术形态** | 单页 SPA + FastAPI 单进程 + AkShare/Tushare + SQLite |
| **适用场景** | 比赛演示、快速开发排期、答辩材料 |

**画像建立 → 模型适配与选股 → 解读与交互 → 模型自优化**

本系统为比赛演示原型，仅用于技术方案展示与投资研究辅助，不构成任何投资建议。

# 摘　要

本文面向 A 股个人投资者，设计一套可在 3 天内开发完成的 AI 智能选股 MVP。系统以现有 Python 量化计算框架为代码底座，保留因子计算、筛选、组合与回测的成熟结构，替换数据层为 A 股数据源（AkShare / Tushare），重写界面层为单页应用，并补四块能力：**AI 多轮对话建立画像**（不设问卷路径）、**按画像适配 A 股多因子选股**、**基于事实的个股解读与追问**、**AI 参数调优与可视化的版本管理**。

架构采用轻量前后端分离：后端 FastAPI 单进程（同时托管静态页与 API），前端原生 JS + SSE 流式交互，对话与调参过程实时推送，避免整页刷新。A 股因子数据通过 AkShare（免费、无需 token）为主、Tushare（需 token、字段更全）为辅的双适配器接入，并建立本地缓存。方案明确删除冗余兜底：AI 或数据不可用即返回明确错误并提示重试；自优化只调参数不写代码，并以「AI 调参台」可视化面板展示参数演化与指标对比，追求美观、抓眼、可验证，不伪造 AI 推理细节。

**关键词**：AI Agent；A 股选股；多因子；SSE；可视化调参；设计模式

# 目　录

**摘　要** ..... 2

**1 项目定位与增量范围** ..... 6

1.1 定位与既有基础 ..... 6
1.2 四项核心能力 ..... 6
1.3 四个关键设计决策 ..... 7
1.4 三天后的验收标准 ..... 7
1.5 与完整版的关系 ..... 7

**2 三天开发计划** ..... 8

2.1 开工前置（半天内完成） ..... 8
2.2 每日任务分解 ..... 8
2.3 三条排期纪律 ..... 9
2.4 改动范围清单（评审可核对） ..... 9
2.5 交付物清单 ..... 10

**3 精简架构与技术选型** ..... 11

3.1 架构原则 ..... 11
3.2 架构与部署 ..... 11
3.3 技术选型 ..... 12
3.4 数据流与接口 ..... 13
3.5 运行方式与配置 ..... 14

**4 核心用例** ..... 15

4.1 用例图 ..... 15
4.2 关键用例描述 ..... 17
4.3 边界与约束 ..... 17

**5 组件设计** ..... 18

5.1 组件图 ..... 18
5.2 组件职责 ..... 19
5.3 代码目录结构（增量部分） ..... 19
5.4 接口契约 ..... 20

**6 核心流程** ..... 21

6.1 顺序图 ..... 21
6.2 会话与状态 ..... 22
6.3 异常处理（不降级） ..... 22

**7 设计模式（6 个真正落地的）** ..... 23

7.1 选型总览 . . . 23
7.2 策略 + 注册表 . . . 23
7.3 适配器 . . . 23
7.4 门面 . . . 24
7.5 校验链 . . . 24
7.6 观察者/发布-订阅（SSE + 可视化） . . . 25
7.7 命令 + 备忘录 . . . 25
7.8 落地清单与工作量 . . . 26

**8 A 股因子与数据获取 . . . 27**

8.1 因子体系（10 个） . . . 27
8.2 数据获取方式（A 股） . . . 27
8.3 A 股回测规则 . . . 28
8.4 评估指标与调优目标 . . . 28
8.5 数据质量与限制 . . . 29

**9 模型自优化 MVP（参数级） . . . 30**

9.1 闭环设计 . . . 30
9.2 触发条件 . . . 30
9.3 调优实现步骤 . . . 30
9.4 可视化事件契约 . . . 31
9.5 版本面板与回滚 . . . 31
9.6 为什么不做代码级自优化 . . . 31

**10 数据、状态与配置 . . . 33**

10.1 存储布局 . . . 33
10.2 数据表设计 . . . 33
10.3 A 股缓存与字段约定 . . . 33
10.4 配置项 . . . 34
10.5 数据口径与合规 . . . 34

**11 AI 调参可视化设计 . . . 35**

11.1 目标与原则 . . . 35
11.2 页面布局 . . . 35
11.3 视觉与动效规范 . . . 36
11.4 数据驱动与状态机 . . . 36
11.5 实现要点与工作量 . . . 37
11.6 与其他页面的关系 . . . 37

**12 风险、异常与演示 . . . 38**

12.1 风险清单 . . . 38
12.2 异常处理原则 . . . 38
12.3 三分钟演示脚本 . . . 38

12.4 提交前检查清单..... 39
12.5 答辩要点..... 39

# 1 项目定位与增量范围

## 1.1 定位与既有基础

本方案把项目收敛为一个 **A 股、单机、可现场演示**的 AI 选股 MVP，开发周期 3 天。系统以现有 Python 量化计算框架为代码底座：因子标准化、综合打分、组合构建与回测的结构可以直接复用，但**数据层与因子口径必须改为 A 股**，界面层整体重写。

| 现有能力 | 复用方式 | 本次改造 |
| --- | --- | --- |
| 因子计算框架（winsorize、z-score、方向对齐） | 复用计算框架与代码结构 | 因子从 6 个扩展到 10 个 A 股口径 |
| 筛选与加权打分 | 复用排序与权重结构 | 按画像映射权重，支持调优候选注册 |
| 组合构建（等权/风险平价） | 直接复用 | 增加 A 股涨跌停、停牌、T+1 约束 |
| 回测引擎（IS/OOS、多指标） | 复用指标与流程 | 数据换 A 股、按 A 股交易规则重算 |
| 数据获取 | 重写 | AkShare 为主、Tushare 为辅，本地 parquet 缓存 |
| 界面 | 重写 | 单页应用（原生 JS + SSE + ECharts） |

## 1.2 四项核心能力

| 能力 | 现状 | 3 天要做成什么 |
| --- | --- | --- |
| AI 对话画像 | 无 | 多轮对话抽取 6 项画像槽位、主动追问，确认后落库；**不设问卷兜底** |
| 画像驱动的 A 股选股 | 有固定权重预设 | 画像映射到 10 因子权重，输出 Top N、组合权重与 OOS 指标 |
| 解读与追问 | 无 | 基于因子贡献与回测指标生成解读，支持多轮追问，强制风险提示 |
| 参数调优与可视化 | 无 | 指标衰减时搜索因子权重，生成候选版本；「AI 调参台」可视化展示参数演化、指标对比与版本回滚 |

## 1.3 四个关键设计决策

**其一，轻量前后端分离。** 后端 FastAPI 单进程，前端单页应用；逻辑分离、部署不分离：一个 uvicorn 进程同时托管页面与 API，无 Nginx、无独立前端构建、无消息队列。代价约半天到一天，换来多轮对话的流式体验、服务端会话状态与可测试的接口。

**其二，画像只走 AI 对话。** AI 主动追问补全槽位并给出风险等级，用户确认后写入 SQLite；不做问卷与规则的并行路径。

**其三，删除兜底机制。** 不写模板解读、不做降级展示。AI 或数据失败时后端返回明确错误（HTTP 5xx / SSE error），前端提示重试；系统宁可报错，也不展示可能误导用户的内容。

**其四，自优化只调参数并可视化。** 允许 AI 搜索因子权重、生成候选版本，但不改代码；调参过程通过「AI 调参台」实时可视化——展示的是真实的事件流与可验证的指标，不做虚假推理表演。

## 1.4 三天后的验收标准

其一，用户通过多轮对话建立画像，AI 能主动追问缺失项；其二，发起选股后能看到 A 股 Top N、因子贡献、组合权重与 OOS 回测指标；其三，能生成有因子依据的解读并进行多轮追问；其四，能完整演示“指标衰减 → AI 调参 → 可视化演化 → 候选版本 → 人工启用 → 回滚”；其五，AI 或数据不可用时页面明确报错，而不是悄悄降级。

## 1.5 与完整版的关系

完整版设计（81 页、11 张图、16 个设计模式）保留为方案储备，用于回答“后续怎么演进”。本 MVP 是其最小可运行子集：保留分层与接口思想，不做多服务、不做代码级自优化、不做 RAG 与灰度发布；二期可按完整版路线逐步补齐。

# 2 三天开发计划

## 2.1 开工前置（半天内完成）

A 股数据接入是本次最容易被低估的工作量，必须在 D1 前完成验证。AkShare 免费无需 token，作为主数据源；Tushare 需注册获取 token，作为字段补充。股票池建议先固定为**沪深 300 + 中证 500 约 800 只**，避免全市场 5000 只拖慢回测与演示。

| 前置项 | 验收方式 | 预计耗时 |
| --- | --- | --- |
| A 股数据可用 | AkShare 拉取沪深 300 成分股日线与财务指标各一次 | 1 小时 |
| 数据缓存脚本 | 生成 `data/a_share/prices.parquet`、`fundamentals.parquet`、`industry.csv`、`index.parquet` | 1.5 小时 |
| LLM Key（OpenAI 兼容） | 一次调用返回结构化 JSON | 0.5 小时 |
| 接口契约冻结 | REST + SSE 端点与字段写进文档（见 5.4） | 1 小时 |
| 演示口径 | 固定股票池、截止日期、提示词版本、免责声明 | 0.5 小时 |

## 2.2 每日任务分解

| 天 | 时段 | 任务（具体到文件） | 当日验收标准 |
| --- | --- | --- | --- |
| D1 | 上午 | 后端骨架：`backend/main.py`（FastAPI + 静态托管）、`backend/db.py`（SQLite 建表）、`backend/data/`（AkShare/Tushare 适配器 + 缓存） | 页面可访问，`/api/health` 正常；数据缓存生成成功 |
| D1 | 下午 | `backend/factors.py`：10 因子计算与标准化；`backend/agent.py`：LLM 客户端 + 多轮对话 + 画像槽位抽取与追问；`POST /api/sessions/{id}/messages`（SSE） | 连续对话 3 轮抽全 6 项槽位；10 因子可在 800 只股票上算完 |
| D1 | 晚上 | 前端最小可用：`frontend/index.html`、`app.js`、`style.css`，对话区接收 SSE 流式文本 | 页面可完成完整对话，**SSE 链路当天必须打通** |
| D2 | 上午 | `backend/selection.py`：因子筛选、组合构建、OOS 回测（A 股规则）；`POST /api/sessions/{id}/selection` + SSE 进度 | 页面点击选股后能看到 A 股 Top N、权重与回测指标 |

| 天 | 时段 | 任务（具体到文件） | 当日验收标准 |
|---|---|---|---|
| D2 | 下午 | AI 解读与追问：基于因子贡献与指标生成解读、合规校验、流式输出；画像确认与修改接口 | Top N 每只有解读，追问 2 轮有效，敏感表述被拦截 |
| D2 | 晚上 | 前端结果页：Top N 表、因子雷达图、组合饼图、净值曲线（ECharts） | 结果页完整可展示，刷新页面不丢会话 |
| D3 | 上午 | `backend/tuner.py`：10 因子权重搜索（复用回测）、SSE 推送每次 trial、候选版本写入；版本启用/回滚接口 | 一键调优能生成候选版本与新旧指标对比 |
| D3 | 下午 | 「AI 调参台」可视化：前端三栏面板（AI 操作流 / 参数演化 / 指标对比），事件流驱动动画；异常与联调 | 调参过程有动画、参数与指标真实变化、可启用/回滚 |
| D3 | 晚上 | 演示排练两遍、录屏、材料打包、代码冻结 | 3 分钟演示连续两遍无故障 |

## 2.3 三条排期纪律

第一，**每天 18:00 冻结可运行版本**并打 tag，晚上只在冻结版本上做小增量。第二，**Day 3 不再新增功能**，只修 P0 缺陷与打磨演示。第三，**A 股数据先跑通再谈因子**：D1 上午必须完成数据缓存与 10 因子全量计算，否则 D2 无法联调；演示只用缓存数据，不做现场拉取。

## 2.4 改动范围清单（评审可核对）

| 类型 | 文件 | 说明 |
|---|---|---|
| 新增 | `backend/main.py` | FastAPI 入口、路由、SSE、静态托管 |
| 新增 | `backend/db.py` | SQLite：sessions / messages / profiles / versions / audit_logs |
| 新增 | `backend/data/akshare_adapter.py`、`tushare_adapter.py` | A 股行情、财务、行业、指数适配器 |
| 新增 | `backend/factors.py` | 10 因子计算、标准化、行业中性化 |
| 新增 | `backend/agent.py` | 多轮对话、画像抽取、解读、合规校验 |
| 新增 | `backend/selection.py` | 筛选、组合、回测 API 包装（A 股交易规则） |
| 新增 | `backend/tuner.py` | 权重搜索、trial 事件、候选版本 |
| 新增 | `frontend/` | SPA：对话、结果、解读、**AI 调参台** |
| 新增 | `data/a_share/` | 本地缓存（价格/财务/行业/指数） |

| 类型 | 文件 | 说明 |
| --- | --- | --- |
| 保留 | `archive/` 中完整版设计 | 作为二期演进参考 |

> 规模预估：后端约 1100 行，前端约 900 行（含调参台可视化），数据缓存脚本约 200 行；两名开发者在 3 天内可完成，D3 下午留半天缓冲。

## 2.5 交付物清单

| 交付物 | 形式 | 责任人 |
| --- | --- | --- |
| 可运行代码（backend/ + frontend/ + data 缓存脚本） | Git 分支 / zip | 后端 + 前端 |
| 本设计说明书（精简版） | DOCX + PDF | 全员 |
| 5 张图（含 AI 调参台视觉稿） | PNG + 源脚本 | 设计 |
| 3 分钟演示视频 | MP4 | 前端 |
| 答辩问答 10 题 | Markdown | 全员 |

# 3 精简架构与技术选型

## 3.1 架构原则

**逻辑分离、部署不分离**——前后端在代码与接口上分离，但同一个进程托管静态页与 API，不引入 Nginx 与多服务；**数据本地化**——A 股行情、财务、行业与指数全部缓存到本地 parquet，演示不依赖实时接口；**计算内核稳定**——因子、筛选、组合与回测沿用现有框架结构，只替换数据口径与交易规则；**流式优先**——对话、选股进度、调参 trial 统一走 SSE；**失败即报错**——AI 或数据不可用时返回明确错误，不做降级。

## 3.2 架构与部署

如图 1 所示，系统分三层。**前端层**是单页应用（原生 JS + EventSource + ECharts CDN），含对话画像、选股结果、AI 解读与追问、AI 调参可视化四个区块。**应用层**是单进程 FastAPI（uvicorn :8000）：`api-server` 负责路由、会话、SSE 与静态托管；`agent` 负责多轮对话、画像抽取、解读与合规校验；`factor-engine` 负责 10 因子计算、标准化、行业中性化与 Top N 筛选；`backtest-engine` 负责 IS/OOS 回测与指标；`tuner` 负责权重搜索与候选版本；`store` 负责 SQLite 持久化。**数据与外部层**为 `data/app.db`、A 股 parquet 缓存、AkShare/Tushare 数据源与 LLM API。

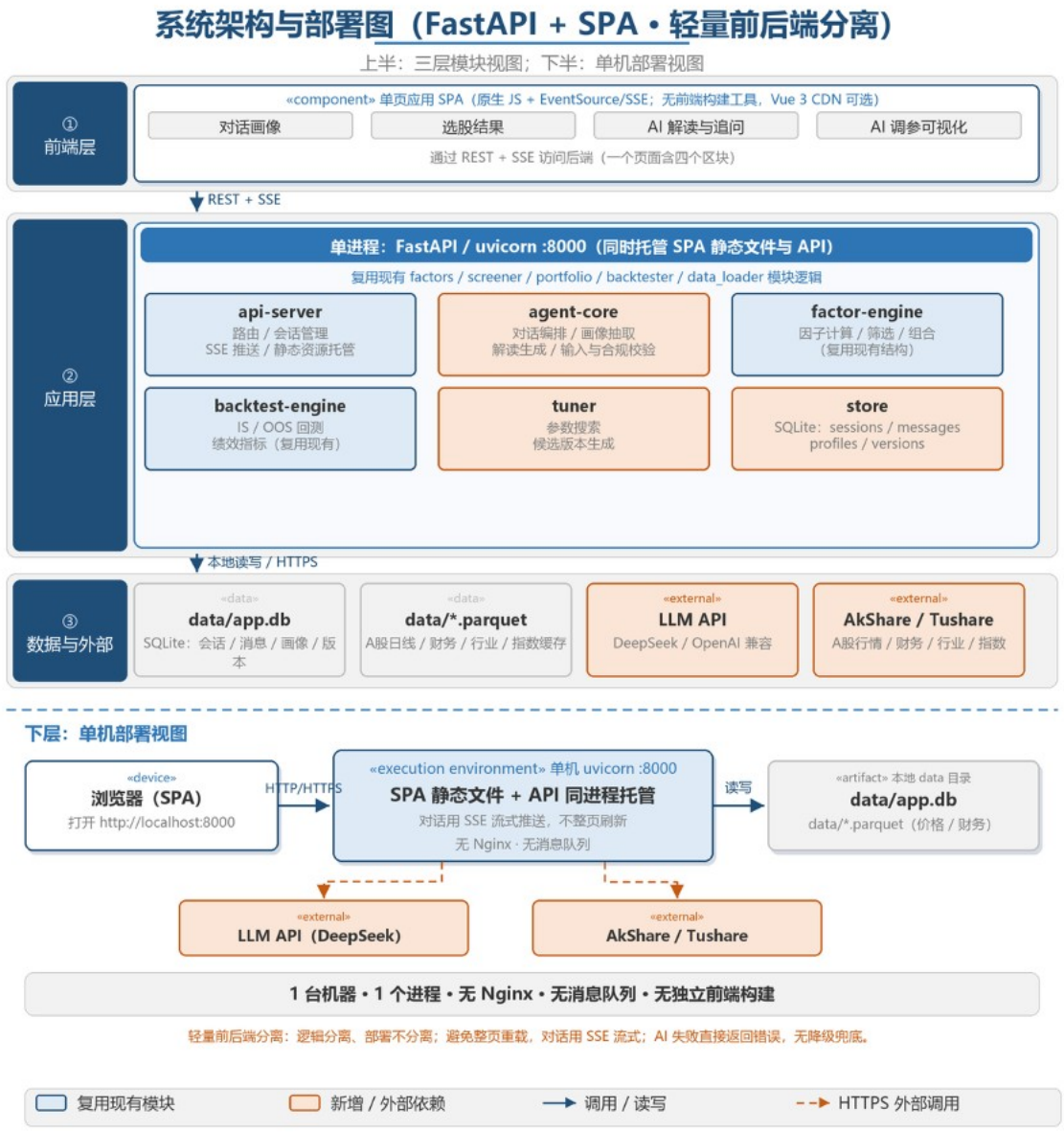



**图 1 系统架构与部署图（FastAPI + SPA • A 股 AI 选股）**

部署上只有一个进程：uvicorn 同时提供 `/api/*` 与静态页面，浏览器首次加载后通过 `EventSource` 保持 SSE；无 Nginx、无独立前端构建、无消息队列。A 股数据由离线缓存脚本预先拉取，服务运行期只读本地文件，既避免接口限流，也保证现场速度。

## 3.3 技术选型

| 技术域 | 选型 | 说明 |
| --- | --- | --- |
| 前端 | 原生 JS + HTML/CSS + ECharts CDN | 无构建工具；图表与调参动画直接用 ECharts 与 CSS 过渡 |
| 通信 | REST + SSE | 对话 token、选股进度、调参 trial 均流式推送 |

| 技术域 | 选型 | 说明 |
| --- | --- | --- |
| 后端 | FastAPI + uvicorn + Pydantic | 异步、自动校验、内置 OpenAPI 文档 |
| A 股数据（主） | **AkShare** | 免费无需 token；覆盖日线、财务指标、行业分类、指数 |
| A 股数据（辅） | **Tushare Pro** | 需 token；字段更全（daily_basic、fina_indicator 等），用于补充 |
| 计算 | pandas / numpy / scipy（沿用现有因子与回测结构） | 因子与回测代码主要复用，改口径不改框架 |
| 存储 | SQLite（stdlib）+ parquet 缓存 | 单文件数据库，随项目拷贝即迁移 |
| 可视化 | ECharts（雷达、条形、净值、散点、仪表盘）+ CSS 动画 | 调参台的核心表现层 |
| 部署 | `uvicorn backend.main:app`（可 Docker 单容器） | 一条命令启动 |

## 3.4 数据流与接口

A 股数据流：离线脚本通过 AkShare/Tushare 拉取沪深 300 + 中证 500 的日线、财务指标、行业分类与基准指数 → 清洗复权后写入 `data/a_share/*.parquet` → 服务运行期只读缓存。交互数据流：浏览器 POST 消息 → FastAPI 写会话 → Agent 调 LLM 抽取画像或生成解读 → SSE 流式回传 → 用户确认画像后触发选股 → 因子引擎读缓存计算 10 因子与 Top N → 回测引擎计算 OOS 指标 → Agent 基于事实生成解读 → SSE 推送结果；调优时 tuner 每完成一组参数就推送一条 trial 事件，前端「AI 调参台」实时可视化。

主要接口（完整契约见 5.4）：

| 方法 | 路径 | 用途 |
| --- | --- | --- |
| POST | `/api/sessions` | 创建会话 |
| POST / GET | `/api/sessions/{id}/messages` | 发送消息（SSE 流式） / 拉取历史 |
| GET / PUT | `/api/sessions/{id}/profile` | 读取 / 确认修改画像 |
| POST | `/api/sessions/{id}/selection` | 创建选股任务 |
| GET | `/api/sessions/{id}/selection/events` | 订阅选股进度与结果（SSE） |
| POST | `/api/sessions/{id}/tune` | 启动参数调优任务 |
| GET | `/api/sessions/{id}/tune/events` | 订阅调参 trial 事件（SSE，驱动调参台） |

| 方法 | 路径 | 用途 |
| --- | --- | --- |
| GET | `/api/versions` | 模型版本列表 |
| POST | `/api/versions/{id}/activate` | 启用版本 |
| POST | `/api/versions/{id}/rollback` | 回滚版本 |

## 3.5 运行方式与配置

启动分两步：先运行 `python backend/data/build_cache.py --universe csi800` 生成 A 股缓存（首次约 10–20 分钟），再运行 `uvicorn backend.main:app --port 8000`。环境变量 `DEEPSEEK_API_KEY` 缺失时启动检查直接报错退出；`TUSHARE_TOKEN` 仅在选用 Tushare 补充字段时需要。前端无需安装依赖，浏览器访问 `http://localhost:8000` 即可。

# 4 核心用例

## 4.1 用例图

如图 2 所示，系统参与者为投资者、AI Agent 与行情数据源（AkShare/Tushare）。共 12 个用例：UC1 对话建立画像、UC2 确认/修改画像、UC3 发起选股、UC4 查看推荐与解读、UC5 追问、UC6 触发参数调优、UC7 查看/启用/回滚版本、UC8 查看因子与回测（现有能力复用），AI 侧 UC9 多轮追问补全画像、UC10 生成个股解读、UC11 搜索候选参数并评估，外部 UC12 提供 A 股行情财务数据。

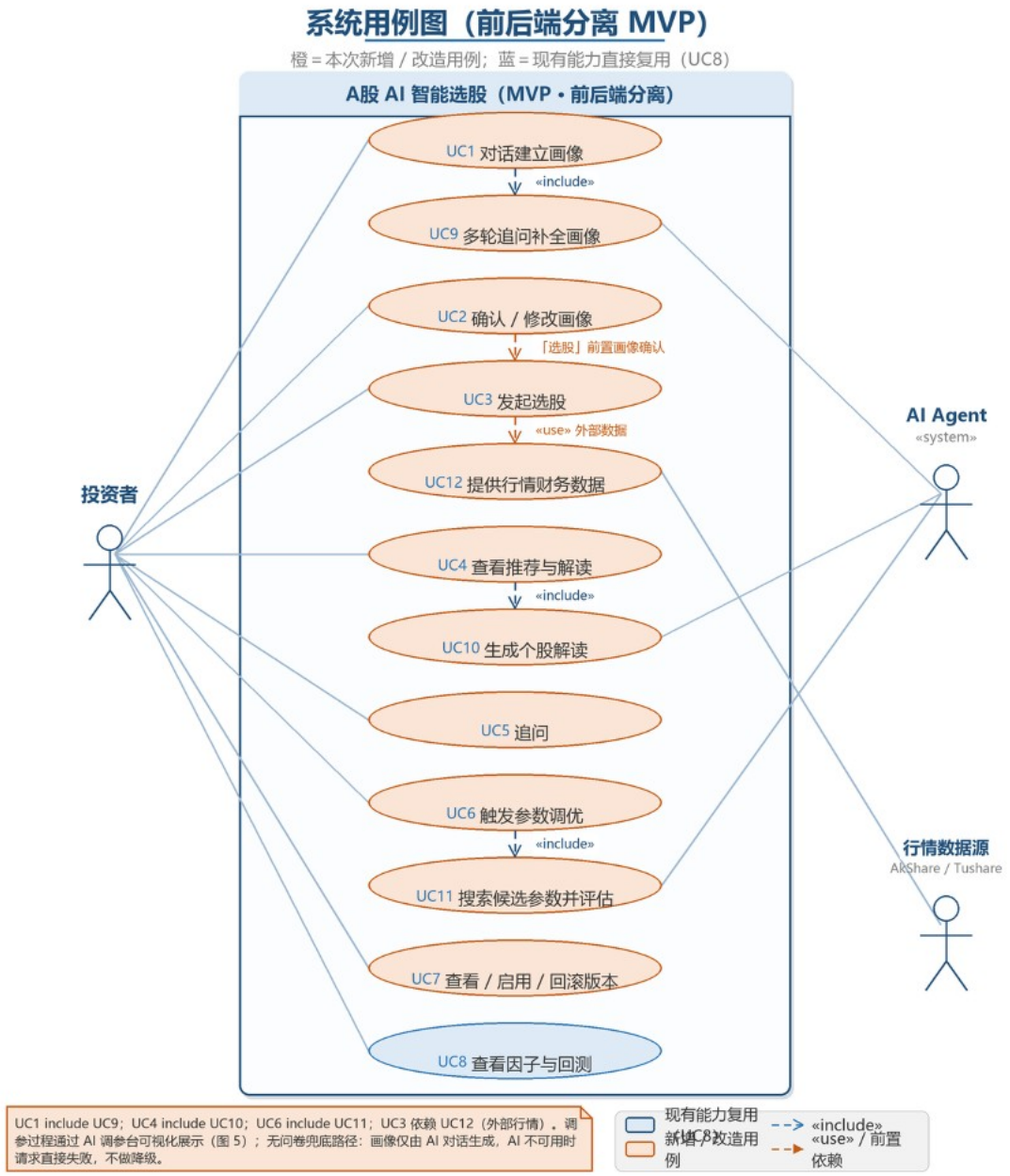



**图 2 系统用例图（A 股 AI 选股 MVP）**

用例关系：UC1 包含（include）UC9，画像必须由 AI 多轮追问完成，没有问卷或规则旁路；UC4 包含 UC10；UC6 包含 UC11；UC3 依赖外部 UC12 的 A 股数据。画像未确认时前端禁用“开始选股”。UC6 的执行过程通过「AI 调参台」可视化呈现（图 5），作为该用例的展示形态而非独立用例。

## 4.2 关键用例描述

| 用例 | 前置条件 | 主流程 | 异常 |
| --- | --- | --- | --- |
| UC1/UC9 对话建画像 | 已创建会话；LLM 可用 | 用户自然语言描述（市场偏好、期限、回撤容忍等）→ AI 每轮最多追问 2 个缺失项 → 槽位齐全后生成画像卡（风险等级、期限、回撤容忍、收益目标、行业禁忌、经验）→ 用户确认 | LLM 超时/5xx → SSE `error` → 前端提示“AI 服务不可用，重试”；**不降级问卷** |
| UC3/UC4/UC10 选股与解读 | 画像已确认；A 股缓存已生成 | 创建任务 → 10 因子计算与 Top N → 组合权重（等权/风险平价）→ OOS 回测 → AI 基于因子贡献与指标生成解读 → SSE 推送 → 前端渲染 | 数据缺失或计算异常 → 任务 failed；解读失败 → 已算结果可展示并提示“解读生成失败，可重试” |
| UC5 追问 | 已有选股结果 | 用户追问“为什么选它/风险在哪” → AI 携带会话历史与结果摘要回答 → 流式返回 | 上下文超限 → 截断旧消息并提示；LLM 失败 → 错误提示 |
| UC6/UC11 调优与可视化 | 有生效版本；回测可用 | 前端进入「AI 调参台」→ 触发调优 → 每完成一组参数推送 trial 事件（权重、指标、目标值）→ 候选版本与指标对比 → 用户启用或回滚 | 调优失败/超时 → 任务 failed 并保留原版本；不自动切换 |

## 4.3 边界与约束

画像对话的边界是“3–8 轮”：3 轮内应抽到风险等级与期限，超过 8 轮仍未完整则 AI 在回复中列出缺失项请用户补充。解读的边界是“只解释已有事实”：不引用外部新闻、不预测涨跌、不给买卖点。调优的边界是“只调参数”：只允许 10 个因子权重与档位，越界参数直接拒绝。A 股约束单独处理：停牌股剔除、涨跌停不可成交、T+1 与交易成本计入回测。

> 关于可视化的一条原则：AI 调参台展示的每一个数字都来自真实 trial 事件与回测结果，步骤文案由后端状态机生成；它可以被美化、加动画，但**不允许编造 AI 的“思考内容”或虚假指标**。美而不假，是这类演示面板的底线。

# 5 组件设计

## 5.1 组件图

如图 3 所示，系统共 8 个内部组件：web-app（SPA）、api-server（FastAPI）、agent-core（对话/画像/解读/校验）、factor-engine（10 因子计算与筛选）、backtest-engine（回测与指标）、tuner（参数搜索与候选版本）、store（SQLite）、data-adapter（AkShare/Tushare 适配）。外部依赖两个：LLM API 与 A 股数据源。

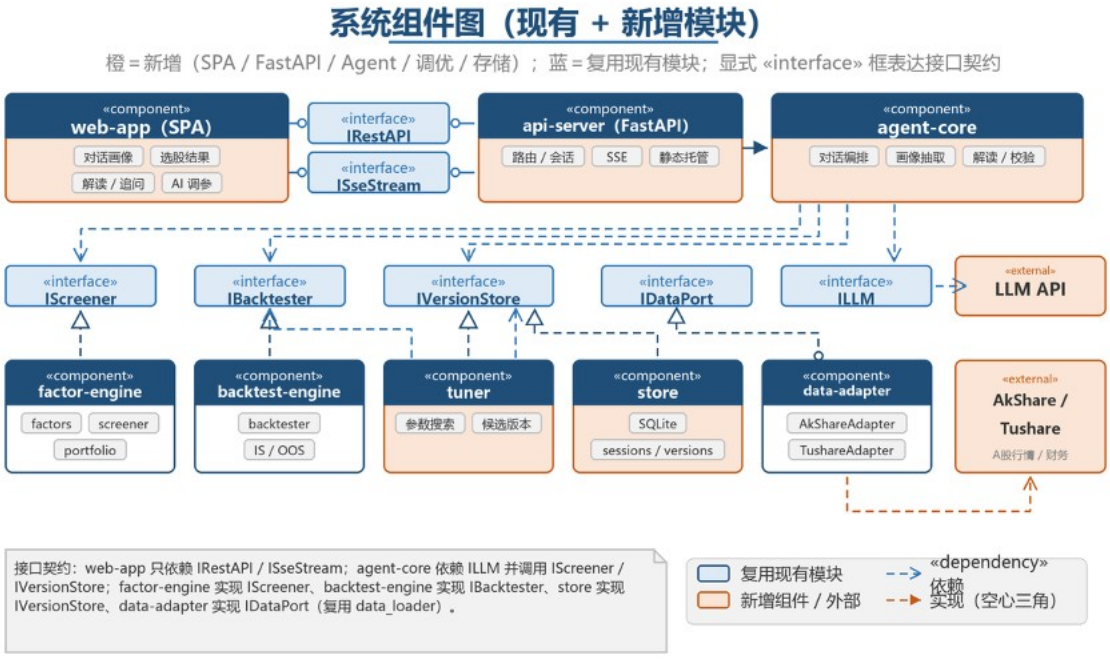



**图 3 系统组件图（A 股 • 现有 + 新增模块）**

依赖关系：web-app 只依赖 IRestAPI 与 ISseStream；api-server 依赖 agent-core、factor-engine、backtest-engine、tuner 与 store；agent-core 依赖 ILLM 与 store；tuner 依赖 IBacktester 与 IVersionStore；factor-engine 实现 IScreener，backtest-engine 实现 IBacktester，data-adapter 实现 IDataPort，store 实现 IVersionStore。前后端只通过 HTTP + SSE 通信。

## 5.2 组件职责

| 组件 | 类型 | 职责 | 关键接口 |
|---|---|---|---|
| web-app（frontend/） | 新增 | 单页界面：对话、画像卡、结果、解读与追问、**AI 调参台** | IRestAPI、ISseStream |
| api-server（backend/main.py） | 新增 | 路由、会话、SSE、静态托管、请求校验 | IRestAPI |
| agent-core（backend/agent.py） | 新增 | 多轮对话编排、画像抽取、解读生成、合规校验 | ILLM |
| factor-engine（backend/factors.py） | 改造复用 | 10 因子计算、标准化、行业中性化、Top N 筛选 | IScreener |
| backtest-engine（backend/selection.py） | 复用改造 | A 股规则回测（T+1、涨跌停、停牌、成本）、OOS 指标 | IBacktester |
| tuner（backend/tuner.py） | 新增 | 因子权重搜索、trial 事件推送、候选版本生成 | IBacktester、IVersionStore |
| store（backend/db.py） | 新增 | SQLite：会话、消息、画像、版本、审计 | IVersionStore（其余为内部方法） |
| data-adapter（backend/data/） | 新增 | AkShare/Tushare 双适配器 + parquet 缓存读取 | IDataPort |

## 5.3 代码目录结构（增量部分）

```
stock-screener/
├── backend/
│   ├── main.py             # FastAPI 入口、路由、SSE、静态托管
│   ├── db.py               # SQLite 建表与读写
│   ├── agent.py            # LLM 客户端、多轮对话、画像抽取、解读、合规
│   ├── factors.py          # A 股 10 因子计算与标准化
│   ├── selection.py        # 筛选、组合、A 股回测指标
│   ├── tuner.py            # 权重搜索、trial 事件、候选版本
│   ├── data/
│   │   ├── akshare_adapter.py
│   │   ├── tushare_adapter.py
│   │   └── build_cache.py  # 离线拉取 A 股缓存（prices/fundamentals/industry/index）
│   └── schemas.py          # Pydantic 请求/响应模型
├── frontend/
│   ├── index.html          # 单页：对话 / 结果 / 解读 / AI 调参台
│   ├── app.js              # 会话、SSE、渲染
│   ├── charts.js           # ECharts 封装（雷达、条形、净值、散点）
│   └── style.css           # 深色仪表盘样式与动画
├── data/a_share/           # prices/fundamentals/industry/index parquet
└── requirements.txt        # 新增 fastapi、uvicorn、akshare、tushare、requests
```

## 5.4 接口契约

前后端以本节为唯一契约，D1 上午冻结，双方并行开发。

| 接口 | 请求 | 响应 |
| --- | --- | --- |
| `POST /api/sessions` | — | `{session_id, created_at}` |
| `POST /api/sessions/{id}/messages` | `{content}` | SSE：`token`、`slots`、`profile_ready`、`error` |
| `GET /api/sessions/{id}/messages` | — | 历史消息列表 |
| `GET / PUT /api/sessions/{id}/profile` | 画像槽位与确认状态 | 画像 JSON |
| `POST /api/sessions/{id}/selection` | `{top_n, universe}` | `{task_id}` |
| `GET /api/sessions/{id}/selection/events` | `?task_id=` | SSE：`progress`、`result`、`error` |
| `POST /api/sessions/{id}/tune` | `{trials, search_space?}` | `{job_id}` |
| `GET /api/sessions/{id}/tune/events` | `?job_id=` | SSE：`trial`、`best_update`、`done`、`error` |
| `GET /api/versions` | — | 版本列表（权重、指标、状态） |
| `POST /api/versions/{id}/activate` / `rollback` | `{reason?}` | 操作结果与当前生效版本 |

`trial` 事件是调参台的驱动数据，字段固定为：`trial_index`、`weights`、`metrics{cagr,sharpe,mdd,ir}`、`objective`、`is_best`。前端收到后更新条形图、雷达、散点与指标卡，不轮询、不猜测。

# 6 核心流程

## 6.1 顺序图

如图 4 所示，一次完整交互覆盖“对话画像 → A 股选股与回测 → 解读与追问 → 调优与可视化”四个阶段。参与者自左向右为：用户、SPA、FastAPI、Agent、LLM API、因子引擎、回测引擎、SQLite、数据适配层（AkShare/Tushare）。

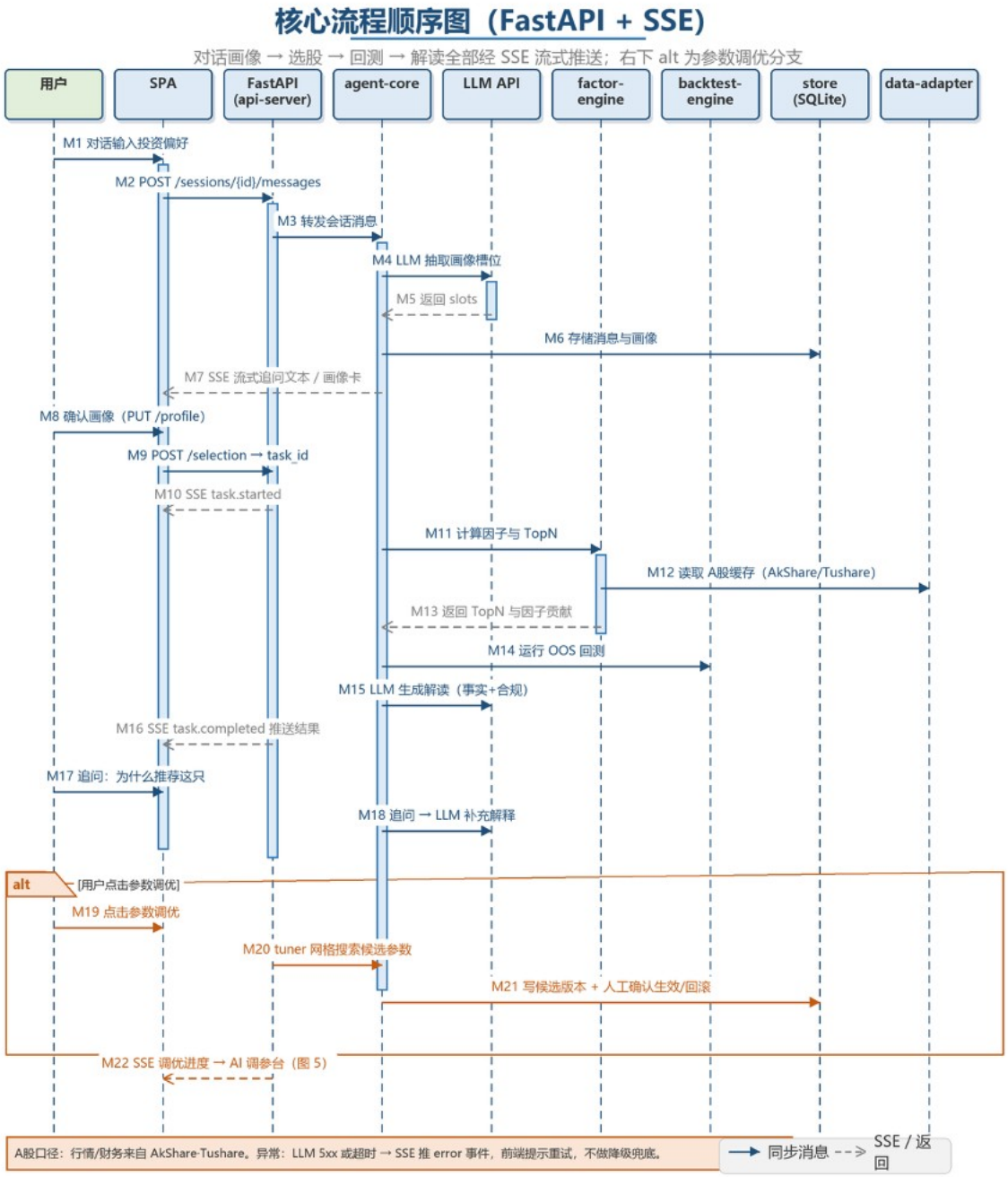



**图 4 核心流程顺序图（FastAPI + SSE）**

关键点有三。第一，**对话流式化**：用户每次发言以 POST 发送，FastAPI 写入消息后调用 Agent，LLM 文本增量通过 SSE `token` 事件持续推回页面，用户看到 AI 逐字追问而不是整段刷新。第二，**事实与表达分离**：因子分、组合权重与回测指标全部由确定性代码产出，Agent 只负责组织语言并通过合规校验。第三，**调优旁路 + 可视化**：用户点击“参数调优”后，`tuner` 复用回测引擎逐组评估权重，每完成一组就推送 `trial` 事件；前端调参台实时更新参数条形图、雷达图、散点与指标卡，全部结束后推送 `done` 与候选版本；用户在版本面板启用或回滚。

## 6.2 会话与状态

多轮对话上下文保存在服务端：`sessions` 记录会话，`messages` 记录每轮消息，`profiles` 保存槽位与确认状态；前端只保存 `session_id`（localStorage），刷新页面后重新拉取历史消息与画像，不依赖浏览器内存中的业务状态。SSE 采用“一次请求一条流”：对话、选股、调优各自独立，事件结束即关闭；断线后前端最多重连 3 次，失败则显示错误与手动重试按钮。

## 6.3 异常处理（不降级）

| 场景 | 后端行为 | 前端表现 |
| --- | --- | --- |
| LLM 超时 / 5xx / 限流 | 记录日志，SSE 发 `error` 并结束流；HTTP 场景返回 502 | 弹出“AI 服务不可用，请重试”，保留输入可重发 |
| LLM 返回 JSON 解析失败 | 重试 1 次后返回 `error`（不猜测、不拼接） | 提示“AI 输出格式异常，请重试或换一种说法” |
| A 股数据缺失 / 计算异常 | 选股任务标记 failed，SSE `error` 携带原因 | 显示失败原因与重试按钮；不展示为成功结果 |
| A 股停牌 / 涨跌停导致无法成交 | 回测按规则剔除或按下一可成交价处理，结果页标注 | 结果中展示“已剔除 N 只不可成交标的” |
| 调优搜索失败 | 任务 failed，原生效版本不变 | 调参台停止动画并提示“调优失败，继续使用当前版本” |
| 画像未确认就请求选股 | 返回 409 与提示 | 禁用按钮并引导先确认画像 |

> 所有异常都走同一条链路：后端写日志 → SSE 推 `error`（或 HTTP 5xx）→ 前端明确提示并提供重试。系统不提供“降级成功”的返回，也不在失败时展示半成品结果。

# 7 设计模式（6 个真正落地的）

## 7.1 选型总览

3 天项目只保留 6 个能直接降低改动风险的模式，每个在代码里都有明确落点；其余模式在完整版中讨论，本期不引入。

| 模式 | 解决的问题 | 现有基础 | 本次新增 |
| --- | --- | --- | --- |
| 策略 + 注册表 | 因子权重随画像切换、调优候选可注册 | 权重预设表与自定义权重入口 | `WEIGHT_PRESETS` 注册表 + 取权函数 |
| 适配器 | AkShare/Tushare 双数据源可替换 | 数据获取函数 | `IDataPort` + `AkShareAdapter/TushareAdapter` |
| 门面 | LLM 调用散落路由，难测试 | 无 | `AgentFacade`（对话/画像/解读/校验） |
| 校验链 | 输入、JSON、合规逐层拦截 | 无 | `GuardChain` 3 个校验节点 |
| 观察者/发布-订阅 | 对话 token、选股进度、调参 trial 需实时推送 | 无 | `SseHub` + 前端调参台订阅 |
| 命令 + 备忘录 | 调优需可执行、可撤销、可审计 | 回测能力已有 | `TuneCommand` + `VersionSnapshot` |

## 7.2 策略 + 注册表

10 因子权重按画像映射，调优候选也需要动态注册。注册表让“权重”成为一类可替换策略，避免在筛选函数里堆 if/else。

```
# backend/factors.py（追加约 15 行）
WEIGHT_PRESETS = {"conservative": CONSERVATIVE_W, "balanced": BALANCED_W,
                  "aggressive": AGGRESSIVE_W, "candidate": None}

def register_preset(name: str, weights: dict) -> None:
    if abs(sum(weights.values()) - 1.0) > 1e-6 or any(w < 0 for w in weights.values()):
        raise ValueError(f"权重非法: {weights}")
    WEIGHT_PRESETS[name] = dict(weights)

def get_weights(name: str) -> dict:
    return WEIGHT_PRESETS.get(name, BALANCED_W)
```

## 7.3 适配器

A 股数据有两个真实来源：AkShare（免费、无需 token）与 Tushare（字段全、需 token）。适配器让业务代码只依赖统一端口，换源或测试时不必改因子引擎。

```
# backend/data/base.py + akshare_adapter.py + tushare_adapter.py（约 40 行）
class IDataPort(Protocol):
    def daily(self, codes: list[str], start: str, end: str) -> pd.DataFrame: ...
    def fundamentals(self, codes: list[str]) -> pd.DataFrame: ...
    def industry(self) -> pd.DataFrame: ...
    def index_daily(self, code: str, start: str, end: str) -> pd.DataFrame: ...

class AkShareAdapter:        # 主数据源：免费、无需 token
    def daily(self, codes, start, end): return ak.stock_zh_a_hist(...)
    def fundamentals(self, codes): return ak.stock_financial_analysis_indicator(...)

class TushareAdapter:        # 辅助：字段更全，按需启用
    def daily(self, codes, start, end): return pro.daily(...).rename(columns=...)
```

## 7.4 门面

路由不拼提示词、不解析 JSON、不处理重试。`AgentFacade` 把 LLM 相关逻辑收敛到 `agent.py`，路由只做校验与 SSE 转发。

```
# backend/agent.py（核心约 30 行）
class AgentFacade:
    def __init__(self, llm, store, guard):
        self.llm, self.store, self.guard = llm, store, guard

    async def chat(self, session_id: str, content: str):
        history = await self.store.recent_messages(session_id, limit=20)
        slots = await self.llm.json("profile_extract", {"history": history, "content":
content})
        self.guard.validate_profile(slots)           # 校验失败直接抛错，不降级
        await self.store.save_slots(session_id, slots)
        async for token in self.llm.stream("reply", {"slots": slots, "content": content}):
            yield token                               # 路由层转发为 SSE token

    async def explain(self, facts: dict, question: str = "") -> str:
        text = await self.llm.text("explain", {"facts": facts, "question": question})
        return self.guard.check_compliance(text)
```

## 7.5 校验链

LLM 输出与用户输入都不可信：结构校验 → 合规检查 → 字段约束，任一失败即抛异常，由接口层转成错误事件。

```
# backend/agent.py（GuardChain 约 25 行）
class GuardChain:
    BANNED = ("稳赚", "必涨", "保证收益", "推荐买入", "满仓")

    def validate_profile(self, slots: dict) -> None:
        required = ("risk_level", "horizon", "drawdown_tolerance")
        missing = [k for k in required if not slots.get(k)]
        if missing:
            raise ValueError(f"画像缺少必填项: {missing}")
```

```
    def check_compliance(self, text: str) -> str:
        hits = [w for w in self.BANNED if w in text]
        if hits:
            raise ValueError(f"合规拦截: {hits}")
        return text
```

## 7.6 观察者/发布-订阅（SSE + 可视化）

后端把对话 token、选股进度、调参 trial 统一抽象为事件，通过 `SseHub` 分发给订阅的 HTTP 连接；前端 `EventSource` 订阅后更新界面，其中 `trial` 事件直接驱动「AI 调参台」的图表与动画。这是观察者模式在前后端分离架构下的自然落地，也是可视化“真实感”的来源。

```
# backend/main.py（SseHub 约 20 行）
class SseHub:
    def __init__(self):
        self._subscribers: dict[str, list[asyncio.Queue]] = {}

    def subscribe(self, channel: str) -> asyncio.Queue:
        q = asyncio.Queue()
        self._subscribers.setdefault(channel, []).append(q)
        return q

    async def publish(self, channel: str, event: str, data: dict):
        for q in list(self._subscribers.get(channel, [])):
            await q.put((event, data))
```

## 7.7 命令 + 备忘录

调优变更是“可执行、可撤销的操作”；执行前保存生效版本快照，即备忘录。版本面板的启用/回滚都走这两个方法。

```
# backend/tuner.py（约 25 行）
@dataclass
class VersionSnapshot:
    version_id: str
    weights: dict
    metrics: dict
    status: str = "candidate"

class TuneCommand:
    def __init__(self, store, weights: dict, metrics: dict):
        self.store, self.weights, self.metrics = store, weights, metrics
        self._backup = None

    def apply(self) -> str:
        self._backup = self.store.active_version()        # 备忘录
        return self.store.save_version(
            VersionSnapshot(uid(), self.weights, self.metrics, "active"))

    def undo(self) -> None:
```

```
        if self._backup:
            self.store.restore_version(self._backup)      # 回滚
```

## 7.8 落地清单与工作量

| 文件 | 内容 | 预计代码量 | 工时 |
|---|---|---|---|
| backend/factors.py | 权重注册表、10 因子、标准化 | ~350 行 | 1 天 |
| backend/data/ | AkShare/Tushare 适配器 + 缓存脚本 | ~250 行 | 1 天 |
| backend/agent.py | 门面、LLM 客户端、校验链、提示词 | ~450 行 | 1.5 天 |
| backend/db.py | SQLite 表与读写、版本快照 | ~180 行 | 0.5 天 |
| backend/tuner.py | 权重搜索、trial 事件、命令与快照 | ~180 行 | 0.5 天 |
| backend/main.py | 路由、SSE、静态托管 | ~280 行 | 0.5 天 |
| frontend/ | 单页界面 + AI 调参台（ECharts + CSS 动画） | ~900 行 | 1.5 天 |

> 六个模式合计新增代码约 2600 行（含前端），符合 3 天预算。模式服务于“改得动、测得快、演示好看”，而不是为了模式而模式。

# 8 A 股因子与数据获取

## 8.1 因子体系（10 个）

A 股散户占比高、短期反转与换手率效应明显、小市值长期有溢价、财报质量因子波动大，因此采用 10 个因子的组合，兼顾价值、质量、成长、动量、反转、波动、流动性与规模。

| # | 因子 | 计算口径 | 方向 | 预期逻辑 |
|---|---|---|---|---|
| F1 | 价值 EP | 1 / 市盈率（TTM） | 越高越好 | 便宜的公司长期占优 |
| F2 | 价值 BP | 1 / 市净率（LF） | 越高越好 | 低估值修复 |
| F3 | 质量 ROE | 近 4 季 ROE（TTM） | 越高越好 | 盈利能力 |
| F4 | 质量现金流 | 经营现金流 / 净利润 | 越高越好 | 盈利质量 |
| F5 | 成长 | 近 4 季营收同比增速 | 越高越好 | 景气度 |
| F6 | 动量 60 日 | 近 60 交易日收益率 | 越高越好 | 中期趋势 |
| F7 | 反转 5 日 | 近 5 交易日收益率 | **越低越好** | 短期超买回调 |
| F8 | 低波 60 日 | 近 60 日日收益年化波动率 | **越低越好** | 低波动异象 |
| F9 | 流动性 | 近 20 日日均成交额（对数） | **越低越好** | 流动性溢价 |
| F10 | 规模 | ln（自由流通市值） | **越低越好** | 小市值溢价 |

所有因子统一处理：按方向对齐 → 去极值（1%/99% 分位）→ 行业中性化（按申万一级行业分组 z-score）→ 标准化。综合得分 `Score_i = Σ_f w_f · z_{i,f}`，权重和为 1。

## 8.2 数据获取方式（A 股）

数据分四类：行情、财务、行业分类、基准指数。以 AkShare 为主（免费、无需 token），Tushare 为补充（字段全、需 token）。

| 数据类别 | 关键字段 | AkShare 接口（主） | Tushare 接口（辅） | 频率/时点 |
|---|---|---|---|---|
| 股票列表 | 代码、名称、上市日、状态 | `stock_info_a_code_name`、`stock_zh_a_spot_em` | `stock_basic` | 日频，收盘后 |

| 数据类别 | 关键字段 | AkShare 接口（主） | Tushare 接口（辅） | 频率/时点 |
|---|---|---|---|---|
| 日线行情 | 开高低收、成交量/额、复权因子 | `stock_zh_a_hist`（qfq/hfq） | `daily`、`adj_factor` | 日频，收盘后 |
| 财务指标 | ROE、营收/净利、PE、PB、经营现金流 | `stock_financial_analysis_indicator`、`stock_a_indicator_lg` | `fina_indicator`、`daily_basic`、`income` | 财报披露 / 日频估值 |
| 行业分类 | 申万一级行业 | `stock_board_industry_name_em`、`stock_board_industry_cons_em` | `index_classify`、`index_member` | 周频更新 |
| 基准指数 | 沪深 300、中证 500 收盘价 | `stock_zh_index_daily_em`、`index_zh_a_hist` | `index_daily` | 日频 |
| 交易日历 | 交易日、停复牌 | `tool_trade_date_hist_sina`、`stock_tfp_em` | `trade_cal`、`suspend_d` | 日频 |

**获取策略**：离线脚本 `build_cache.py` 一次性拉取近 5 年数据并落盘 parquet；服务运行期只读缓存，不调用外部接口。财务指标按“公告日”对齐到行情（避免未来函数），财报未披露时沿用上一期并在数据中记录 `report_date` 与 `announce_date`。

## 8.3 A 股回测规则

回测必须体现 A 股交易制度，否则结果不可信：**T+1**（当日买入次日才能卖出）、**涨跌停不可成交**（收盘价封板则按下一可成交价处理）、**停牌剔除**、**双边交易成本**（佣金万分之三 + 印花税千分之一，卖出单边）、**复权价格**（前复权用于信号，后复权用于收益）、调仓周期默认 20 个交易日。基准为沪深 300，股票池为沪深 300 + 中证 500。

## 8.4 评估指标与调优目标

沿用现有回测指标：CAGR、Sharpe、最大回撤 MDD、信息比率 IR、月胜率。调优目标函数为：

$$J = Sharpe_{OOS} - 0.5 \times |MDD_{OOS}| + 0.2 \times IR_{OOS}$$

并约束候选在训练集（IS）上的 $CAGR_{IS}$ 不低于基线的 80%，防止过拟合。调优搜索空间为 10 个因子权重（$\Sigma w = 1$，单因子 $\in [0, 0.4]$，步长 0.02），随机采样 60–120 组；每组回测结果通过 SSE `trial` 事件推送前端，驱动「AI 调参台」可视化。

## 8.5 数据质量与限制

三类风险必须如实说明：其一，**幸存者偏差**——股票池使用当前成分股，历史退市股未纳入；其二，**财务数据滞后**——季报滞后 1–3 个月，回测按公告日对齐但仍有近似；其三，**数据源差异**——AkShare 与 Tushare 的字段口径可能略有差异，缓存中记录来源与抓取时间。所有输出仅为研究辅助，不构成投资建议。

# 9 模型自优化 MVP（参数级）

## 9.1 闭环设计

自优化只做到参数级，复用回测引擎，成本最低、可演示性最强：**监控**（记录当前版本 OOS 指标）→ **触发**（用户点击或指标越界）→ **搜索**（10 因子权重随机采样 60–120 组）→ **对比**（候选与生效版本指标）→ **可视化**（每次 trial 实时推送前端调参台）→ **人工确认与发布/回滚**。代码级自优化（LLM 改代码 + 沙箱）本期不做，仅保留接口位。

## 9.2 触发条件

| 指标 | 触发条件（示例） | 数据来源 | 动作 |
| --- | --- | --- | --- |
| 样本外夏普 | 较生效版本下降超过 30% | 回测引擎 | 前端提示“建议调优” |
| 最大回撤 | 超过画像约束（如 20%） | 回测引擎 | 提示并高亮风险 |
| 信息比率 | 低于 0 且连续 2 次回测 | 回测引擎 | 同上 |
| 人工触发 | 演示/答辩需要 | 调参台按钮 | 立即启动调优任务 |

触发后调用 `POST /api/sessions/{id}/tune`，任务在 FastAPI `BackgroundTasks` 中执行，每组参数完成即向 `tune` SSE 通道推送一条 `trial` 事件。

## 9.3 调优实现步骤

`backend/tuner.py` 约 180 行：读取当前生效权重 → 在约束内随机采样 60–120 组 → 逐组调用回测（A 股规则）→ 计算目标函数 `J = Sharpe_OOS - 0.5×|MDD_OOS| + 0.2×IR_OOS` → 每组完成后 `hub.publish("tune", "trial", {...})` → 全部完成后取最优且通过 IS 稳定性检查的一组，写入 `versions` 表（`status=candidate`）→ 推送 `best_update` 与 `done` → 前端调参台展示候选版本并允许启用/回滚。

版本表（SQLite）：

```
CREATE TABLE versions (
  id TEXT PRIMARY KEY,
  parent_id TEXT,
  weights_json TEXT NOT NULL,
  metrics_json TEXT NOT NULL,
```

```
  status TEXT NOT NULL,              -- candidate / active / archived
  source TEXT,                       -- auto_tune / manual
  created_at TEXT NOT NULL
);
```

## 9.4 可视化事件契约

调参台不是“动画表演”，而是由真实事件驱动。事件字段固定，前端据此更新图表：

| 事件 | 触发时机 | 关键字段 |
| --- | --- | --- |
| tuning_started | 任务开始 | job_id、trials_total、base_weights、base_metrics |
| trial | 每组参数回测完成 | trial_index、weights、metrics{cagr,sharpe,mdd,ir}、objective、is_best |
| best_update | 出现更优候选 | best_weights、best_metrics、improvement |
| done | 搜索结束 | candidate_version_id、compare{current,candidate}、duration_ms |
| error | 任一阶段失败 | code、message |

事件速率控制：若回测很快（< 20ms/组），后端按 ≥100ms 间隔节流推送，避免前端动画被刷爆；前端对刻度与数字做 300ms 过渡，视觉上呈现“参数在演化”的连续过程。

## 9.5 版本面板与回滚

前端调参台底部与「模型版本」区域共用版本数据：当前生效版本、候选版本列表、指标对比、启用/回滚按钮。启用走 `POST /api/versions/{id}/activate`，服务端先把当前版本保存为快照再切换；回滚走 `rollback`，恢复快照并将候选标记为 `archived`。所有操作写 `audit_logs`，演示时可展示“一次错误参数被回滚”的审计记录。

> 设计取舍：不自动发布。调优只生成候选，由人决定是否启用；这不是兜底，而是权限边界——不可逆的模型变更必须由人确认，才敢现场演示。

## 9.6 为什么不做代码级自优化

代码级自优化需要隔离沙箱、AST 白名单、单元测试、防未来函数校验四件基础设施，每件都不止半天工作量，现场风险也高。本期用参数级调优 + 可视化已能讲清“模

型根据表现自我改进”的叙事，性价比最高。二期可按完整版第 7 章补齐补丁生成、沙箱验证与发布审批。

# 10 数据、状态与配置

## 10.1 存储布局

系统只保留两类存储：SQLite 负责业务状态，parquet 负责 A 股数据缓存。二者都在 `data/` 下，随项目拷贝即可迁移，无数据库服务。

```
data/
├── app.db                      # SQLite：sessions/messages/profiles/versions/audit_logs
├── a_share/
│   ├── prices.parquet          # A 股日线（前/后复权）
│   ├── fundamentals.parquet    # 财务指标（含 report_date / announce_date）
│   ├── industry.csv            # 申万一级行业分类
│   ├── index.parquet           # 沪深 300、中证 500 基准
│   └── universe.csv            # 股票池（沪深 300+中证 500）与状态
└── meta.json                   # 缓存元信息：数据源、抓取时间、字段版本
```

## 10.2 数据表设计

| 表 | 关键字段 | 说明 |
|---|---|---|
| `sessions` | id、created_at、updated_at | 会话；前端 localStorage 只存 session_id |
| `messages` | id、session_id、role、content、tokens、created_at | 多轮对话流水 |
| `profiles` | id、session_id、slots_json、risk_level、confirmed、updated_at | 画像槽位与确认状态 |
| `versions` | id、parent_id、weights_json、metrics_json、status、source、created_at | 模型版本快照 |
| `audit_logs` | id、actor、action、from_version、to_version、reason、created_at | 启用/回滚/调优审计 |

画像 `slots_json` 至少包含 6 项：风险等级、投资期限、回撤容忍、收益目标、行业禁忌、投资经验。后端保存前做 Schema 校验，缺项直接拒绝。

## 10.3 A 股缓存与字段约定

`prices.parquet` 为宽表（行=交易日，列=股票代码），保存前复权收盘价用于信号、后复权收盘价用于收益计算；`fundamentals.parquet` 以 `(code, report_date)` 为主键，额外保存 `announce_date` 用于防止未来函数；`industry.csv` 记录申万一级行业，用

于行业中性化；`index.parquet` 保存沪深 300 与中证 500 的日线。`meta.json` 记录数据来源（AkShare/Tushare）、抓取时间与字段版本，便于复现与审计。

## 10.4 配置项

| 配置 | 方式 | 说明 |
| --- | --- | --- |
| `DEEPSEEK_API_KEY` | 环境变量 | 启动时校验，缺失即启动失败并打印明确报错（不降级） |
| `TUSHARE_TOKEN` | 环境变量 | 选用 Tushare 补充字段时必填 |
| `LLM_MODEL` / `LLM_BASE_URL` | 环境变量 | 默认 DeepSeek 官方地址与模型 |
| `DATA_DIR` | 环境变量，默认 `data/` | SQLite 与 parquet 缓存位置 |
| 股票池 / 调仓周期 / Top N | 请求参数或前端表单 | 演示固定沪深 300+中证 500、20 日调仓、Top 10 |
| 提示词版本 | 代码常量 `PROMPT_VERSION` | 每次改动写日志，便于复现 |

## 10.5 数据口径与合规

A 股数据口径为前复权价格、公告日对齐的财务指标；已知限制（幸存者偏差、财报滞后、数据源差异）在文档与页面中如实说明。解读输出强制附加“不构成投资建议”，不允许出现收益承诺、个股买卖点与满仓建议。会话数据仅存本机 `data/app.db`，演示前可重置该文件恢复到干净状态，不收集真实个人信息。

# 11 AI 调参可视化设计

## 11.1 目标与原则

当 AI 搜索因子权重时，用户需要“看见模型在变好”，而不仅是等一个结果。本章设计一个**美观、抓眼、可验证**的「AI 调参台」：它不要求精确展示 AI 的推理过程，也不播放与真实状态无关的假动画，而是把**真实的参数演化与指标变化**用高质感的仪表盘呈现出来。三条原则：**真实驱动**——每个数字来自 `trial/best_update/done` 事件；**视觉优先**——深色仪表盘、发光卡片、平滑过渡，适合投屏与录屏；**克制表达**——不伪造“AI 内心想法”，只用状态标签与可核对的指标讲故事。

## 11.2 页面布局

调参台为单页三栏布局（1440×900 设计稿，1280 以上可用），如图 5 所示。




**图 5 AI 调参可视化面板（示意）**

- **顶部状态栏**：标题「AI 调参台 · 模型自动优化中」、进度环（已完成 trial / 总 trial）、当前阶段胶囊（诊断 / 搜索 / 评估 / 建议）。

- **左栏「AI 操作流」**（约 300px）：竖向时间轴，5 个步骤卡片依次为「诊断：动量因子衰减」「生成候选参数」「回测 64/120 组」「对比新旧指标」「建议启用候选 v3」；当前步骤高亮 + 脉冲圆点 + 打字机光标，已完成步骤打勾，未来步骤灰化。
- **中栏「参数演化」**（约 560px）：上方 10 因子权重条形图，当前 vs 候选双色对比、候选条发光；下方左雷达图（当前/候选两条多边形），右散点图（120 个候选点按目标值由蓝到橙着色，最优点放大 + 脉冲光圈）。
- **右栏「指标对比」**（约 480px）：4 张指标卡（Sharpe、年化、最大回撤、IR），数字大号等宽并带涨跌箭头；下方候选 vs 当前净值曲线（渐变填充）；底部版本时间轴（v1 → v2 → 候选 v3）。
- **底部操作条**：左「启用候选版本」、右「回滚」，并标注「所有变更需人工确认 · 历史版本可回滚」。

## 11.3 视觉与动效规范

配色采用深色科技感：背景 `#0B1220 → #111C2E` 渐变，卡片 `#151F33` + 1px `#2A3A55` 描边，主色青蓝 `#22D3EE`、强调橙 `#F59E0B`、成功绿 `#34D399`、风险红 `#FB7185`，正文浅灰 `#CBD5E1`。字体用系统无衬线 + 等宽数字（如 IBM Plex Sans/Mono 或系统字体）；数字使用等宽字体避免跳动。

动效只服务于信息表达：数字 300ms 缓动滚动，条形图 300ms 宽度过渡，雷达与散点以 200ms 淡入，最优点高亮脉冲 1.2s 循环；调参完成后「启用」按钮出现一次 600ms 呼吸光效。所有动画可通过 CSS `prefers-reduced-motion` 关闭，投屏性能优先。

## 11.4 数据驱动与状态机

前端只维护一个状态对象：`{jobId, phase, trialDone, trialTotal, baseWeights, bestWeights, bestMetrics, history[]}`。SSE 事件到达后更新状态并触发重绘：`tuning_started` 初始化进度与基线；`trial` 追加到散点、更新条形图与指标卡；`best_update` 更新候选参数多边形与最优高亮；`done` 收起进度环、展示候选版本与启用按钮；`error` 停止动画并显示可重试提示。

左栏文案由后端状态机的真实阶段生成（诊断 → 搜索 → 评估 → 建议），不是前端随机播放；每张步骤卡片可点开查看该阶段的真实事件（如 trial 数、最优目标值）。这样“好看”与“真实”并不冲突。

## 11.5 实现要点与工作量

图表使用 ECharts：条形图（`bar` + 动态 `setOption`）、雷达图（`radar`）、散点图（`scatter` + `visualMap`）、净值曲线（`line` + 渐变 `areaStyle`）、进度环（`gauge` 或 SVG 圆环）。动效用 CSS transition 与 `requestAnimationFrame` 实现数字滚动。性能上限制散点最多 120 个、事件按 ≥100ms 节流、重绘只更新 `setOption` 的增量字段。异常状态保留骨架屏与错误卡片，不显示半成品。

预计工作量约 1 天：布局与样式 0.4 天，ECharts 组件与事件绑定 0.4 天，动效与联调 0.2 天。前端文件集中在 `frontend/charts.js` 与 `frontend/style.css`，不引入额外图表库。

## 11.6 与其他页面的关系

调参台通过顶部导航与「对话画像」「选股结果」「AI 解读」并列，共享同一个 `session_id` 与 SSE 连接管理器；从结果页点击“优化模型”即可跳转并自动启动调优。版本启用/回滚后，返回选股页会用新权重重新计算，形成“看见变化 → 确认变化 → 使用变化”的闭环体验。

# 12 风险、异常与演示

## 12.1 风险清单

| 风险 | 概率 | 影响 | 对策 |
| --- | --- | --- | --- |
| AkShare 接口限流或字段变更 | 中高 | A 股数据拉取失败 | 离线缓存 + `meta.json` 记录版本；Tushare 作为备用字段源；演示只用缓存 |
| Tushare token 未配置 | 中 | 部分财务字段缺失 | 默认只用 AkShare 可得字段，Tushare 仅作可选增强 |
| 财务数据公告日对齐误差 | 中 | 回测未来函数风险 | 按 `announce_date` 对齐，缺失时滞后一期并记录 |
| LLM 限流 / 输出格式异常 | 中 | 对话或解读失败 | Schema 校验 + 重试 1 次，仍失败则明确报错（不降级） |
| 前后端联调拖期 | 中 | 页面无法演示 | D1 冻结契约、D1 晚打通 SSE；前端先用 mock 数据开发 |
| 调参可视化性能 | 低 | 动画卡顿 | 限 120 个 trial、≥100ms 节流、增量 `setOption` |
| 回测耗时 | 中 | 选股/调优变慢 | 股票池限定沪深 300+中证 500；后台任务 + SSE 进度 |
| 演示误操作 | 低 | 状态混乱 | 演示前重置 `data/app.db`，预置干净会话与版本 |

## 12.2 异常处理原则

不降级、不静默、不假装成功。后端对所有外部依赖失败做三件事：记录结构化日志、向 SSE 发送 `event: error`、在 HTTP 场景返回明确 4xx/5xx 与错误码。前端对应三种表现：对话失败保留输入可重发；选股失败展示原因与重试按钮；调优失败停止调参台动画并保留原生效版本。**已算出的确定性结果（因子分、回测指标）可以展示，但必须与“解读生成失败”的提示分开**，不能让用户误以为失败部分也是成功的。

## 12.3 三分钟演示脚本

| 时间 | 画面 | 讲解要点 |
| --- | --- | --- |
| 0:00–0:35 | 对话页：输入“稳健型、3-5 年、能接受 20% 回撤、偏好消费与科技、不碰 ST” | AI 逐字追问并补全画像，展示 SSE 流式对话 |
| 0:35–1:05 | 确认画像 → 开始选股，SSE 进度条 | 强调 A 股股票池与画像驱动权重 |

| 时间 | 画面 | 讲解要点 |
| --- | --- | --- |
| 1:05–1:40 | 结果页：A 股 Top N、10 因子贡献、组合权重、OOS 指标 | 展示 10 因子与 A 股回测规则 |
| 1:40–2:10 | AI 解读与追问：“为什么选它/风险在哪” | 事实来自因子与回测，合规校验拦截敏感表述 |
| 2:10–2:50 | 「AI 调参台」：点击优化 → 参数演化动画 → 指标对比 → 启用候选 → 回滚 | 可视化展示真实 trial 数据与人工闸门 |
| 2:50–3:00 | 收尾 | 历史模拟不代表未来，不构成投资建议 |

演示前额外演练两个异常场景：临时填错 LLM Key 看报错提示；调优任务中途失败看调参台停止并保留原版本。答辩时主动展示异常处理，比只演示成功路径更可信。

## 12.4 提交前检查清单

- [ ] 一条命令启动 FastAPI，页面与 `/api/health` 均可访问
- [ ] A 股缓存生成成功，10 因子可在股票池上算完
- [ ] 多轮对话 ≥5 轮，AI 能主动追问且槽位正确更新
- [ ] 画像确认前选股按钮禁用；确认后选股结果与画像匹配
- [ ] 选股 SSE 进度与最终结果正常，刷新页面不丢会话
- [ ] 解读有因子依据，追问 2 轮有效，敏感表述被拦截
- [ ] AI 调参台能展示真实 trial 参数/指标演化，启用/回滚正确
- [ ] LLM 不可用时页面明确报错并可重试，不出现假结果
- [ ] 3 分钟演示视频已录制；演示数据与提示词版本固定
- [ ] 精简版说明书与 5 张图已导出；`data/app.db` 已重置

## 12.5 答辩要点

评委最可能追问五件事。**A 股因子数据从哪来**：AkShare 为主（免费、无需 token）覆盖日线、财务、行业、指数，Tushare 补充字段；离线缓存保证演示稳定。**调参可视化是否真实**：所有数字来自真实 trial 事件与回测结果，步骤来自后端状态机，没有伪造推理过程。**为什么不做兜底**：降级会掩盖故障、增加测试面、误导用户；失败即明确报错

更可控。**前后端分离是否过度**：只做“逻辑分离、部署不分离”，一个进程托管页面与 API，成本约半天到一天。**自优化是否安全**：只调参数不写代码，样本外评估 + 人工确认 + 快照回滚 + 审计留痕。